

DAT32-92

Machine Learning I — Linear and Logistic (Compact)

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	4
1. From regression to classification	5
1.1. The data we will use	5
2. Logistic regression	7
2.1. The sigmoid function	7
2.2. Interpreting the output and the coefficients	8
2.3. How it is fitted	9
3. The confusion matrix and the core metrics	12
3.1. The confusion matrix	12
3.2. Accuracy and why it lies	12
3.3. Precision and recall	13
3.4. The F1 score	13
4. The decision threshold and the ROC curve	16
4.1. Threshold as a business lever	16
4.2. The ROC curve	17
4.3. Communicating results in plain language	19
5. Multi-class classification	20
5.1. One-vs-rest	20
5.2. Softmax (multinomial logistic regression)	20
5.3. Evaluating multi-class models	22
6. Feature scaling	23
7. Feature engineering for linear models	27
7.1. Polynomial features	27
7.2. Interaction terms	27
7.3. Encoding categorical variables	28
8. Regularization for logistic regression	30
9. When simple models win	32
10. The end-to-end classification pipeline	33
Sources and further reading	35

Preface

In *Intro to Machine Learning* you walked the complete machine-learning cycle once, on linear regression: you defined a target, split the data, beat a baseline, fitted with ordinary least squares and with gradient descent, diagnosed overfitting from learning curves, controlled it with L1 and L2 regularization, estimated performance honestly with k-fold cross-validation, tuned hyperparameters on a validation set, extended to multiple features, and packaged everything in a reproducible scikit-learn pipeline. That same cycle underlies this course. We do not re-teach it; we assume you own it.

What changes here is the **question**. Regression predicts a number — a rent, a demand, a revenue. But most decisions a business actually makes are not “how much?” they are “which one?” and “yes or no?": will this customer churn, is this transaction fraud, should this email reach the inbox, will this loan default. That is **classification**, and it is the task type that dominates applied machine learning.

The model we anchor on is **logistic regression**. The name is misleading — it is a classification model, not a regression one — but it is also a promise: it is linear regression’s twin. It is a weighted sum of features, exactly as before; we simply pass that sum through one function that turns it into a probability. Everything you know about coefficients, gradient descent and regularization transfers almost unchanged.

The genuinely new material is **evaluation**. In regression, “how wrong?” had a clean answer in the units of the target. In classification a single error rate hides distinctions that can be critical: a cancer screen that misses a tumour and one that raises a false alarm are both “wrong,” but they are not equally wrong, and no business treats them as such. The bulk of this book is therefore the classification evaluation toolkit — the confusion matrix, precision and recall, the F1 score, the ROC curve and AUC, and the decision threshold — because choosing and defending the right metric is what decides whether a classifier is useful.

By the end you will be able to do what you cannot do today with regression alone: build a logistic-regression classifier and read its output as a probability and its coefficients as odds; choose and defend an evaluation metric from the cost of a false alarm against the cost of a miss; tune the decision threshold to that cost; carry all of this from two classes to many; scale and engineer features for a linear model and regularize it; judge when this simple model is the right one; and write the whole verdict up as a business-oriented evaluation report a non-technical decision maker can act on.

We close where the course’s judgment lives: feature scaling, feature engineering for linear models, regularization for logistic regression, and the case for keeping a model simple. A running example carries the whole book — **predicting which subscribers to a streaming service will cancel next month** (customer churn) — so that every new tool refers to a decision you can picture.

Every idea is worked in Python with **scikit-learn** — the same toolkit you built pipelines with there — so each concept arrives with the few lines that put it to work, and every printed result on these pages is the actual output of the code beside it. Throughout, `X_train/X_test` hold the feature tables and `y_train/y_test` the 0/1 churn labels (1 = churned), exactly the split discipline you already practised.

1. From regression to classification

Your streaming company wants to know, for each subscriber, whether they will cancel (“churn”) in the coming month, so the retention team can call the ones most at risk. You have last year’s data: for every subscriber, their monthly watch hours, tenure, support tickets, plan price — and whether they ultimately churned. This looks like a regression problem you could solve with the tools from *Intro to Machine Learning*. It is not, and seeing exactly why motivates everything that follows.

Definition 1. Classification is supervised learning where the target is a **discrete category** rather than a continuous number. **Binary classification** has two categories (churn / stay, fraud / legitimate); **multi-class classification** has more than two (which of five plans a customer will pick).

Three differences force new tools.

The output is discrete. The label is “churn” or “stay,” not a number on a continuous scale. If you ran ordinary linear regression on a 0/1 target, the model would happily predict 1.4 or -0.3 — values that mean nothing as a category and cannot be read as anything else either.

We want a probability, not just a verdict. The retention team has budget to call 200 people, not 50 000. They do not want a flat “will churn / won’t churn”; they want each subscriber ranked by **how likely** they are to churn, so they can spend the budget on the riskiest. A good classifier outputs a probability between 0 and 1, and the verdict is a second step.

Example — why a probability beats a verdict. Two subscribers are both flagged “will churn.” One is at 0.96 probability, the other at 0.51. With 200 calls to make and 4 000 flags, you cannot treat them alike — you call the 0.96 first. A model that only emits “yes” or “no” has thrown away the information you most need. This is why classification models are built to emit probabilities and the yes-or-no answer is recovered afterward by a **threshold**.

Errors are asymmetric. In rent prediction, being a little too high and a little too low were roughly equally bad. In classification the two ways of being wrong usually cost very different amounts. Failing to flag a churning who then leaves costs you that customer’s whole lifetime value; flagging a loyal customer who was never going to leave costs you a single retention call — often a hundred times cheaper. Treating these as “one error each” is a business mistake, and the evaluation toolkit exists precisely to respect the difference.

Pitfall. Do not fit ordinary linear regression to a 0/1 label and threshold its output at 0.5. It “works” often enough to be tempting, but its predictions are unbounded (values above 1 and below 0 are not probabilities), it is badly distorted by extreme feature values, and its loss function is the wrong shape for a yes-or-no target. Logistic regression fixes all three at once.

1.1. The data we will use

Every snippet in this book runs on one table of last year’s subscribers, `churn.csv`: four numeric features (tenure in months, monthly watch hours, monthly price, support tickets), two categorical

ones (plan, country), and the label churned. We load it and seal a test set immediately — the same discipline as before.

```
import pandas as pd
from sklearn.model_selection import train_test_split

# churn.csv: one row per subscriber from last year – features plus whether
# they ultimately churned (1 = churned, 0 = stayed).
churn = pd.read_csv("churn.csv")

numeric      = ["tenure", "watch_hours", "monthly_price", "support_tickets"]
categorical   = ["plan", "country"]
X = churn[numeric + categorical]
y = churn["churned"]

# Seal a test set now; every score in this book is read from it once, at the end.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)

print(f"{len(X_train)} train rows, {len(X_test)} test rows")
print(f"churn rate: {y.mean():.3f}")
```

```
4800 train rows, 1200 test rows
churn rate: 0.106
```

About one subscriber in ten churned, so the classes are **imbalanced** — a fact that, as the next chapter shows, wrecks the most obvious way of scoring a classifier.

Exercises

1.1. For five business problems, state whether each is a classification task and, if so, name the positive class and which error (false positive or false negative) is more costly. Justify each from the business context.

Answers. (1) A problem is classification when the target is a discrete category. Check that for each you named the positive class **and** the costlier error: e.g. loan default — positive = “defaults,” a false negative (approving a defaulter) usually dwarfs a false positive (declining a good customer); a house-price task is regression, not classification.

2. Logistic regression

We keep linear regression's engine — a weighted sum of the features — and add one component that converts that sum into a probability. Start from the sum you already know,

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p,$$

where, for our subscriber, x_1 might be watch hours, x_2 tenure, and so on. This z (the **log-odds**, as we will see) ranges over all real numbers — exactly the problem. We need to squash it into $(0, 1)$.

2.1. The sigmoid function

Definition 2. The **sigmoid** (or **logistic**) function maps any real number to the open interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It is increasing, passes through $\sigma(0) = 0.5$, and saturates toward 1 as $z \rightarrow +\infty$ and toward 0 as $z \rightarrow -\infty$.

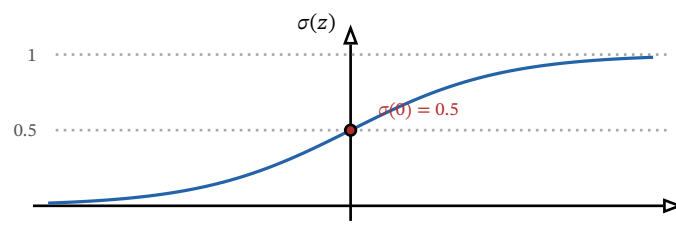


Figure 1: The sigmoid function. It takes the unbounded linear score z and returns a number in $(0, 1)$ we can read as a probability. Large positive z gives near-certainty of the positive class; large negative z , near-certainty of the negative class; $z = 0$ is maximal uncertainty at 0.5.

In code the sigmoid is one line, and it works on a whole array of scores at once:

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

Definition 3. **Logistic regression** models the probability of the positive class as the sigmoid of the linear score:

$$\hat{p} = P(y = 1 \mid x) = \sigma(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p).$$

The fitted $\hat{p}$ is a probability; the predicted class is obtained by comparing $\hat{p}$ to a **decision threshold** (0.5 by default).

2.2. Interpreting the output and the coefficients

The output $\hat{p}$ is read directly: $\hat{p} = 0.87$ means the model assigns this subscriber an 87% chance of churning. That probabilistic reading is one of logistic regression’s great virtues — a graded score a business can rank and act on, not just a yes-or-no answer. Read it primarily as a **ranking score** (0.87 is riskier than 0.60); treat the exact number as an **approximate** probability, because a raw logistic model is not automatically **calibrated** — regularization, class imbalance, and misspecification can pull the absolute values off, so “0.87” need not mean that exactly 87% of such subscribers churn.

The coefficients need a little more care than in linear regression, because the sigmoid sits between z and the probability. Rearranging the definition gives the clean interpretation:

$$\underbrace{\frac{\hat{p}}{1-\hat{p}}}_{\text{odds}} = e^{\beta_0 + \beta_1 x_1 + \dots}, \quad \text{so} \quad \ln\left(\frac{\hat{p}}{1-\hat{p}}\right) = \beta_0 + \beta_1 x_1 + \dots$$

The linear score z is the **log-odds**. So each coefficient β_j is the change in log-odds per one-unit increase in x_j , all else equal; equivalently, e^{β_j} multiplies the **odds**.

To read coefficients as odds we fit on the **raw** (unscaled) features, so one unit is one natural unit — one month, one ticket — and exponentiate:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Fit on the raw (unscaled) numeric features so each coefficient is a change in
# log-odds per one natural unit of that feature.
odds_model = LogisticRegression(max_iter=1000).fit(X_train[numeric], y_train)
odds_ratios = np.exp(odds_model.coef_[0]) # e^{beta_j}: factor on the odds

for name, ratio in zip(numeric, odds_ratios):
    print(f"{name:16s} {ratio:.3f}")
```

```
tenure          0.896
watch_hours     0.953
monthly_price   1.061
support_tickets 6.218
```

Example — reading a churn coefficient. The odds ratio on “support tickets last month” is 6.22: each additional support ticket multiplies the odds of churn by about 6.22, all else equal — support complaints are a strong churn signal. A feature whose odds ratio is 1 (coefficient 0) has no effect; here tenure and watch hours sit just **below** 1, so more of either slightly lowers the odds of churn. The quick read is the ratio’s side of 1: above 1 pushes toward churn, below 1 toward staying.

Pitfall. A logistic coefficient is a change in **log-odds**, not a change in probability. The same one-ticket increase moves the probability a lot near $\hat{p} = 0.5$ (the steep part of the sigmoid) and almost not at all near $\hat{p} = 0.99$ (the flat part). “An extra ticket adds 0.12 to the probability” is wrong — the effect on probability depends on where you start.

2.3. How it is fitted

We cannot use the residual sum of squares here — a 0/1 target with the sigmoid makes RSS a bumpy, hard-to-optimise surface. Logistic regression is fitted by minimising a different loss, **log loss** (cross-entropy), which rewards confident correct probabilities and punishes confident wrong ones.

Definition 4. The **log loss** (binary cross-entropy) for predictions $\hat{p}^{(i)}$ against labels $y^{(i)} \in \{0, 1\}$ is

$$L(\beta) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \ln \hat{p}^{(i)} + (1 - y^{(i)}) \ln(1 - \hat{p}^{(i)})].$$

For a positive example ($y = 1$) only the first term survives, so the loss is $-\ln \hat{p}$: small when $\hat{p}$ is near 1, exploding as $\hat{p} \rightarrow 0$.

This loss is **convex**, so gradient descent — the very method you implemented earlier — slides down to its single global minimum with no local-minimum trouble. And the gradient you descend is the twin of the one you already wrote.

Definition 5. Collect the features (with a leading column of ones for the intercept) into a matrix X and write $\hat{p} = \sigma(X\beta)$ for the vector of predicted probabilities. The gradient of the log loss is then

$$\nabla L(\beta) = \frac{1}{n} X^T (\hat{p} - y).$$

This is **identical in form** to the linear-regression gradient $\frac{1}{n} X^T (\hat{y} - y)$ you already know — the only change is that the prediction $\hat{p}$ is now the sigmoid of the linear score instead of the linear score itself.

Because the gradient has the same shape, so does the training loop: start the coefficients at zero and step downhill. That is the whole model — the sigmoid, this gradient, and the descent loop you already own. On a labelled set the from-scratch coefficients land exactly where scikit-learn's do — here `C=np.inf` switches regularization off, so the library minimises the same bare log loss (Chapter 8 explains C):

```
import numpy as np
from sklearn.linear_model import LogisticRegression

def fit_logistic(X, y, lr=0.3, n_steps=2000):
    Xb = np.column_stack([np.ones(len(X)), X]) # leading 1s column for the intercept
    beta = np.zeros(Xb.shape[1])
    for _ in range(n_steps):
        p = sigmoid(Xb @ beta) # current predicted probabilities
        grad = Xb.T @ (p - y) / len(y) # gradient of the log loss
        beta -= lr * grad # one step downhill
    return beta

# a small synthetic set from known coefficients, to check the fit
rng = np.random.default_rng(0)
```

```

X = rng.normal(size=(400, 2))
z = -0.4 + 1.3 * X[:, 0] - 0.9 * X[:, 1]          # true log-odds
y = (rng.uniform(size=400) < sigmoid(z)).astype(int)

print(fit_logistic(X, y).round(3))                # from-scratch coefficients
sk = LogisticRegression(C=np.inf).fit(X, y)        # C=np.inf: no regularization
print(np.r_[sk.intercept_, sk.coef_[0]].round(3))  # scikit-learn's coefficients

```

```

[-0.32  1.1  -0.638]
[-0.32  1.1  -0.638]

```

In practice you let scikit-learn run this descent for you — with a faster solver and regularization built in — but the machinery underneath is exactly that loop. Using it is the same three-line shape as any scikit-learn model, with one classification-specific habit: ask for `predict_proba`, the probability, and threshold it yourself rather than letting `predict` decide at 0.5 for you. Because our table mixes numeric and categorical columns, we wrap the model in the standard preprocessing you will meet in full in Chapter 6 and Chapter 7 — scaling the numbers, one-hot encoding the categories — so it can consume the whole table; treat that wrapper as a black box until then.

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

# One pipeline: scale the numeric columns, one-hot the categorical ones, then fit
# logistic regression — every step fitted on the training data only.
pre = ColumnTransformer([
    ("num", StandardScaler(), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=1000))])
model.fit(X_train, y_train)

p_hat = model.predict_proba(X_test)[:, 1]  # column 1 = P(churn)
y_pred = (p_hat >= 0.5).astype(int)        # apply the default threshold

```

That fitted model, and the probabilities `p_hat` it assigns the sealed test set, are the running model for the next three chapters: every metric, threshold and curve below is computed from those probabilities, not from `y_pred` alone.

Exercises

2.1. Implement logistic regression **from scratch**: code the sigmoid, the log-loss gradient $\frac{1}{n}X^T(\hat{p} - y)$, and a gradient-descent loop. Fit it to a labelled binary dataset and confirm your coefficients match scikit-learn's unregularized fit (`C=np.inf`) to a few decimals. Then interpret three coefficients in terms of odds (e^{β_j}), and explain why the effect of a feature on the **probability** depends on the starting point.

2.2. Plot the sigmoid function and mark $\sigma(0)$. Explain, using the curve, why a one-unit change in a feature moves the predicted probability more near $\hat{p} = 0.5$ than near $\hat{p} = 0.99$.

Answers. **(1)** Once your descent loop has converged it should reproduce scikit-learn's coefficients to a few decimals (the matching-output check in Section 2.3); state your three interpretations as **odds** multipliers e^{β_j} (positive coefficient $\rightarrow e^{\beta_j} > 1$), never as a fixed change in probability — the probability effect is largest near $\hat{p} = 0.5$. **(2)** $\sigma(0) = 0.5$; the slope $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ peaks at $z = 0$ (value 0.25) and vanishes in the tails, so a fixed step in z moves the probability most near $\hat{p} = 0.5$.

3. The confusion matrix and the core metrics

We have probabilities and, after thresholding, predicted classes. Now: is the classifier any good? “What fraction did it get right?” is the first instinct and the first trap. Every classification metric is built from one small table, so we build it first.

3.1. The confusion matrix

Definition 6. For binary classification with a chosen threshold, the **confusion matrix** cross-tabulates predicted class against actual class into four counts:

- **True Positive (TP)**: predicted positive, actually positive.
- **False Positive (FP)**: predicted positive, actually negative — a **false alarm**.
- **False Negative (FN)**: predicted negative, actually positive — a **miss**.
- **True Negative (TN)**: predicted negative, actually negative.

By convention the **positive** class is the event of interest (here, “churn”).

	Pred: churn	Pred: stay
Actual: churn	TP correct catch	FN missed churning
Actual: stay	FP false alarm	TN correct pass

Figure 2: The confusion matrix for churn prediction — actual class on the rows, predicted on the columns, the same axes scikit-learn uses. We list the positive class (churn) first so the true-positive cell sits top-left; scikit-learn instead orders classes numerically (0 = stay first), so its printed matrix leads with “stay” (see the code below). The green diagonal is correct decisions; the red off-diagonal is the two kinds of error: a false negative (FN) is a churning you failed to call, a false positive (FP) a loyal customer you called for nothing. Almost every metric below is a ratio of these four cells.

3.2. Accuracy and why it lies

The first instinct is to ask what fraction of predictions the model got right — and a single example is enough to see why that instinct is dangerous.

Example — the 97%-accurate model that is worthless. Imagine an even more imbalanced case than ours: only 3% of subscribers churn in a given month. A “model” that predicts **stay** for everyone is right on all 97% who stay — 97% correct. Yet it catches zero churners, so the retention team gets an empty call list. That fraction-correct score is an artefact of the class imbalance, not evidence of skill.

That “fraction correct” is **accuracy**, and the example is the warning that comes with it.

Definition 7. Accuracy is the fraction of correct predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

Accuracy is intuitive and, on imbalanced problems, often useless: it rewards the model for the easy majority and stays silent about the rare class you actually care about.

3.3. Precision and recall

The fix is to stop scoring the easy majority and ask two sharper questions about the positive class — and which one you care about depends entirely on which error hurts.

Example — which one the business needs. A **cancer screening** test must not miss a real tumour: a missed case (FN) can be fatal, while a false alarm (FP) costs only a follow-up scan. So it is tuned to **catch every real case**, accepting more false alarms. A **spam filter** faces the opposite: sending one real, important email to the junk folder (FP) is far worse than letting one spam reach the inbox (FN), so it is tuned to **make each flag count**, accepting more misses. The same model, the same maths — opposite priorities, set by the cost of each error.

Those two priorities have names: **recall** for “catch every real case,” **precision** for “make each flag count.”

Definition 8. Precision answers “of those we flagged positive, how many truly were?”:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

Recall (sensitivity, true-positive rate) answers “of all true positives, how many did we catch?”:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

So **precision** is the metric to watch when **acting on a positive is expensive** (the spam filter, guarding against false alarms) and **recall** the one to watch when **missing a positive is expensive** (the cancer screen, guarding against misses).

Pitfall. Precision and recall trade off against each other and you can trivially max out either alone. Flag **everyone** as positive and recall is a perfect 100% (you missed no one) while precision collapses. Flag only your single most certain case and precision may be 100% while recall is near zero. A number quoted without its partner is meaningless — always report precision **and** recall together.

3.4. The F1 score

Precision and recall can each be maxed out alone (the pitfall above), so a single honest summary must combine them in a way neither extreme can fool — and the obvious average is not it.

Example — why harmonic, not arithmetic. The “flag everyone” model has recall 1.0 and precision, say, 0.03. Their **arithmetic** mean is a misleadingly respectable 0.515 — yet the model is useless, so a fair summary should sit near zero. The **harmonic** mean does: $2(0.03 \cdot 1.0)/(0.03 + 1.0) \approx 0.058$, because it is pulled close to the **smaller** of the two numbers.

That harmonic mean of precision and recall is the **F1 score**.

Definition 9. The **F1 score** is the harmonic mean of precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

The harmonic mean is close to the **smaller** of the two, so F1 is high only when **both** precision and recall are high. A model that scores 1.0 on one and 0.0 on the other gets $F1 = 0$, not 0.5.

The four metrics are not four measurements — they are four ratios of the **same** four counts. Watch them all fall out of our model’s test-set matrix.

Example — one matrix, every metric. Our churn model is scored on the sealed test set of 1200 subscribers, of whom 128 truly churn. At the default threshold it produces this confusion matrix:

	Pred. churn	Pred. stay
Actual churn	TP = 62	FN = 66
Actual stay	FP = 28	TN = 1044

Every metric now reads off these four numbers:

$$\text{Accuracy} = \frac{62 + 1044}{1200} = 0.92, \quad \text{Precision} = \frac{62}{62 + 28} = 0.69,$$

$$\text{Recall} = \frac{62}{62 + 66} = 0.48, \quad F1 = 2 \cdot \frac{0.69 \cdot 0.48}{0.69 + 0.48} = 0.57.$$

Read the outcome from the numbers: 92% accuracy looks fine, yet the model catches only **48 of every 100** real churners (recall) and **69 of every 100** it flags are genuine (precision). Accuracy concealed both facts; precision and recall reveal them; F1 combines the two into one honest 0.57.

scikit-learn computes all of this for you. Its `confusion_matrix` uses the same axes as Figure 2 (actual on the rows, predicted on the columns) but orders the classes by label value — 0 = stay before 1 = churn — so the printout leads with the **stay** row and TN lands in the top-left, mirror-imaged from the teaching layout:

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_test, y_pred))
# rows = actual (0 = stay, 1 = churn), columns = predicted, scikit-learn's order

print(classification_report(y_test, y_pred, target_names=["stay", "churn"], digits=2))
```

```
[[1044  28]
 [  66  62]]
```

	precision	recall	f1-score	support
stay	0.94	0.97	0.96	1072
churn	0.69	0.48	0.57	128
accuracy			0.92	1200
macro avg	0.81	0.73	0.76	1200
weighted avg	0.91	0.92	0.92	1200

F1 is the default single-number metric for imbalanced binary problems, but it bakes in an assumption that precision and recall matter **equally**. When they do not — when a miss costs 150 times a false alarm — F1 is the wrong summary and you should go back to reporting precision and recall separately, or weight them deliberately.

Exercises

3.1. On an imbalanced dataset, build the confusion matrix and compute accuracy, precision, recall and F1. Exhibit a trivial majority-class predictor with high accuracy but zero recall, and explain why accuracy is the wrong metric here.

3.2. For two contrasting problems (one where misses are costly, one where false alarms are costly), select and justify precision, recall or F1, and state which class is positive.

Answers. (1) A “predict the majority” model has recall 0 on the rare positive class while its accuracy equals the majority’s share; your four counts must reproduce accuracy, precision, recall and F1 through the chapter’s formulas. (2) Misses costly (disease, fraud) → optimise recall; false alarms costly (marking real email as spam, expensive interventions) → precision; comparable costs → F1.

4. The decision threshold and the ROC curve

So far we thresholded at 0.5 by reflex. But the threshold is an **adjustable setting**, and moving it is the cheapest, most powerful lever in classification — no retraining required.

4.1. Threshold as a business lever

Definition 10. The **decision threshold** t converts a probability into a class: predict positive when $\hat{p} \geq t$. Lowering t flags more cases; raising t flags fewer.

The two metrics do not respond to that move in the same way. Lowering t can only turn negatives into positives, so it can only **add** true positives: recall rises (or holds) as t falls and falls as t rises — a strict, guaranteed law. Precision usually moves the **opposite** way — fewer, more confident flags tend to be purer — but this is only a **tendency**, not a law. Whether each newly flagged case is a true or a false positive decides whether precision ticks up or down, so a precision curve swept across t is typically **jagged**, not smoothly monotone; you may even see precision dip as you raise t . Recall you can promise; precision you can only expect.

Moving the threshold in code is just re-thresholding the **stored probabilities** — no fit in sight:

```
from sklearn.metrics import precision_score, recall_score

print(" t   flagged  precision  recall")
for t in [0.3, 0.5, 0.8]:
    y_t = (p_hat >= t).astype(int)      # same p_hat, different cut
    print(f"{t:.1f}  {y_t.sum():7d}  {precision_score(y_test, y_t):9.2f}  "
          f"{recall_score(y_test, y_t):6.2f}")
```

t	flagged	precision	recall
0.3	140	0.56	0.62
0.5	90	0.69	0.48
0.8	28	0.86	0.19

Example — adjusting the threshold for the retention team. Read the table straight off the run. At $t = 0.5$ the model flags $62 + 28 = 90$ subscribers at 0.69 precision and 0.48 recall. If the team has budget for far fewer calls and wants each to land on a real churner, raise t to 0.8: now it flags only 28 high-certainty cases at 0.86 precision — many fewer wasted calls — but recall falls to 0.19, so most churners slip through. If instead a lost churner is very costly and calls are cheap, lower t to 0.3 to catch more (140 flagged, recall 0.62) at the price of more false alarms (precision 0.56). Same model, same coefficients — the threshold alone re-balances the two errors to fit the budget and the costs.

Pitfall. The default 0.5 threshold is a convention, not a law, and it is rarely optimal on an imbalanced or cost-asymmetric problem. Choosing the threshold is a **business decision** driven by the relative cost of a false positive versus a false negative — not a setting to leave at its default. And like any tuning choice, pick it on validation data, never on the test set.

Remark. The threshold is this course's lever on imbalance, but not the only one you will meet. Two others are worth knowing by name: **class weighting** (scikit-learn's `class_weight="balanced"` makes each rare-class error count for more during fitting) and **resampling** (over- or under-sampling to even the classes before training). Both are standard practice; both are beyond this course, which stays on metric choice and thresholding. If a problem stays hard after tuning the threshold, these are where to look next.

4.2. The ROC curve

A model that has to commit to one threshold hides how it would behave at every other. The ROC curve shows them all at once, which lets you compare **models** independently of any single threshold.

Definition 11. The **ROC curve** (Receiver Operating Characteristic) plots the **true-positive rate** (recall, $TP / (TP + FN)$) against the **false-positive rate** ($FP / (FP + TN)$) as the threshold sweeps from 1 down to 0. Each point is the model at one threshold; the diagonal is a random classifier.

Both the curve and its area come straight from the stored probabilities:

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_test, p_hat)    # one (fpr, tpr) point per threshold
auc = roc_auc_score(y_test, p_hat)               # ranking quality, one number

plt.plot(fpr, tpr, label=f"AUC = {auc:.2f}")
plt.plot([0, 1], [0, 1], "--")                  # the random-classifier diagonal
plt.xlabel("false-positive rate"); plt.ylabel("true-positive rate"); plt.legend()

print(f"AUC = {auc:.2f}")
```

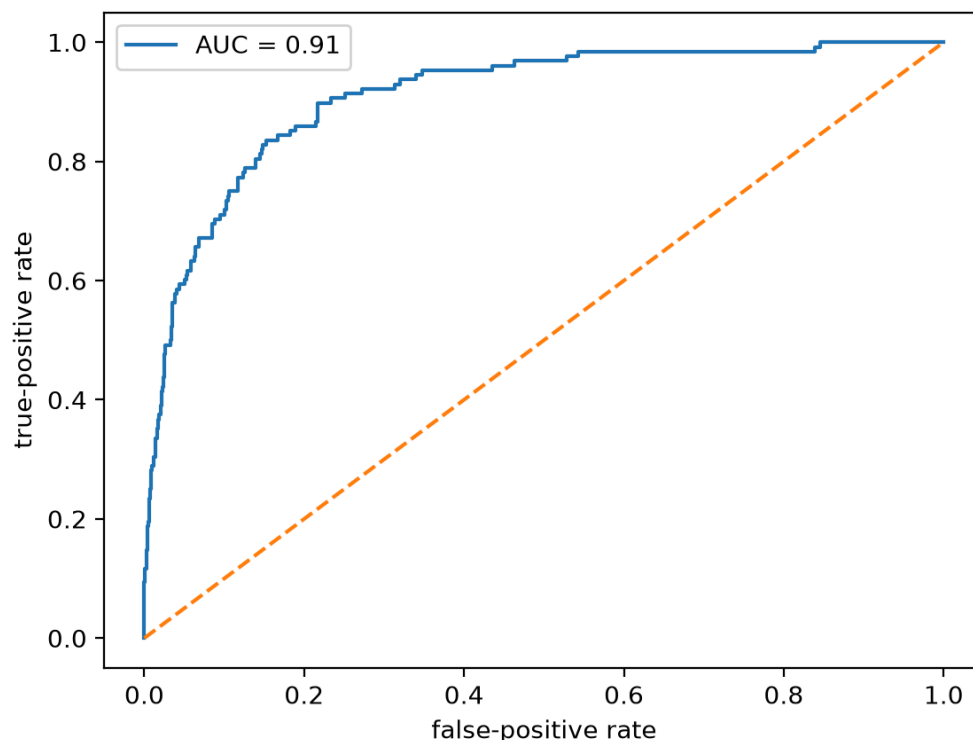


Figure 3: The ROC curve of our churn model on the test set. Sweeping the threshold traces the curve from the top-right (threshold 0: flag everything) to the origin (threshold 1: flag nothing). Because the curve bows hard toward the top-left corner — high recall reached while the false-alarm rate is still low — the model ranks churners well above stayers; the dashed diagonal is the no-skill line a coin-flip would trace. The area under the curve, 0.91 here, summarises the whole picture in one number.

Definition 12. The **AUC-ROC** (Area Under the ROC Curve) summarises the curve as a single number in $[0, 1]$. It equals the probability that the model scores a random positive example higher than a random negative one. $AUC = 1$ is perfect ranking, $AUC = 0.5$ is random, and AUC is **threshold-independent** — it measures how well the model **ranks** cases, separate from where you eventually cut.

Example — AUC compares models, the threshold tunes one. You compare two churn models and model A has AUC 0.88 versus model B's 0.81 — **on average across all thresholds**, A ranks a random churner above a random stayer more often, so as a single summary of ranking quality it is the better model and the one to pick first. Read that as an average, not a guarantee at each cut: two ROC curves can **cross**, so the lower- AUC model may still win at one particular threshold. Having picked A on its AUC , you then choose A's operating threshold from the business costs. AUC is for choosing the model; the threshold is for operating it. Do not confuse the two decisions.

Pitfall. AUC can look reassuringly high on a severely imbalanced problem even when precision at your chosen threshold is poor, because the false-positive rate has a huge negative denominator that hides a lot of false alarms. When positives are rare and precision is what matters, do not stop at AUC : report precision and recall **at the threshold you will actually deploy**. (Practitioners also plot precision directly against recall as the threshold sweeps — the **precision-recall curve** — which, unlike the ROC, is not flattered by a large easy-negative class.)

4.3. Communicating results in plain language

A confusion matrix and a ROC curve are inputs to a **decision** made by people who do not know what recall is. Translate every one of them.

For the **confusion matrix and its metrics**, replace the ratios with counts of people. Instead of “recall is 0.48,” say “of every 100 customers who will actually leave, today’s model catches 48 and misses 52.” Instead of “precision is 0.69,” say “of every 100 customers we call, 69 really were about to leave and 31 were a wasted call.” Then connect the threshold to money: “lowering the cut would catch more leavers but add many more wasted calls — worth it only if a saved customer is worth more than those calls.”

For the **ROC curve and AUC**, do not say “AUC” at all. The AUC is a **ranking** score, so translate it as ranking: “if you put one real leaver and one loyal customer side by side, the model gives the leaver the higher risk score 91 times out of 100.” And read the **curve** as a menu of operating points, not a piece of geometry: “each point on this curve is one possible call list; moving up the curve catches more leavers but wastes more calls, and we get to pick the point that matches the budget.” A manager needs the trade-off the curve offers and the confidence the AUC expresses — never the words “true-positive rate” or “area under the curve.” That translated sentence, not the metric, is the deliverable.

Exercises

- 4.1.** Sweep the decision threshold from 0 to 1 and plot precision and recall against it. Pick a threshold for a stated budget/cost scenario and justify it. Explain why the threshold must be chosen on validation, not test, data.
- 4.2.** Plot a ROC curve for a fitted model and compute its AUC. Compare two models by AUC, then explain why AUC chooses the model while the threshold operates it.
- 4.3.** Take the confusion matrix **and the ROC curve** of a fitted model and write a two-paragraph plain-language report for a non-technical manager. Translate recall and precision into “of 100 real leavers we catch X” and “of 100 we call, Y were right”; translate the AUC as a ranking chance (“shown one leaver and one stayer, the model ranks the leaver higher Z times in 100”); read the ROC curve as a menu of call lists; and state what each error costs and what moving the threshold would do. Use no technical terms — no “recall,” “precision,” “true-positive rate” or “AUC.”

Answers. (1) As t rises, recall falls for certain while precision **tends** to rise — expect a jagged precision curve, not a monotone one, so do not be surprised by a dip. Justify your pick by where extra catches stop being worth the extra false alarms for the stated budget — and choose it on validation, never test. (2) The higher-AUC model ranks better at every threshold, so pick it first, then set the threshold from costs on validation. (3) No jargon: translate recall and precision into “of 100 real leavers we catch X” and “of 100 we call, Y were right,” translate AUC as a ranking chance out of 100, describe the ROC as a set of possible call lists to choose among, state the cost of each error, and say what moving t would buy.

5. Multi-class classification

Churn is binary, but many problems are not: which of five subscription plans will a customer upgrade to, which of a dozen support categories a ticket belongs to. Logistic regression extends to these in two standard ways.

5.1. One-vs-rest

Definition 13. One-vs-rest (OvR, also one-vs-all) trains **one binary classifier per class**, each distinguishing its class from all the others combined. To predict, run all K classifiers and choose the class whose classifier outputs the highest probability.

Example — five plans, five classifiers. For five plans, OvR trains “Basic vs not-Basic,” “Standard vs not-Standard,” and so on — five logistic regressions. A new customer is scored by all five; if the Standard classifier says 0.71 and every other says below 0.5, predict Standard. OvR is simple and reuses the binary model you already have unchanged — in scikit-learn you get it explicitly by wrapping the model in `OneVsRestClassifier`.

5.2. Softmax (multinomial logistic regression)

Definition 14. Softmax regression (multinomial logistic regression) generalises the sigmoid to K classes directly. It computes a linear score z_k per class and converts the whole vector into probabilities that sum to 1:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

The predicted class is the one with the highest probability.

The practical difference: OvR trains K independent models whose probabilities need not be jointly coherent, while softmax fits one model whose K probabilities are guaranteed to form a proper distribution summing to 1. Softmax is usually the cleaner choice when the classes are mutually exclusive; OvR is a robust, simple option when you want one tunable model per class. In scikit-learn, a plain `LogisticRegression` already fits the softmax (multinomial) model for you; OvR is one wrapper away. We fit both on `plans.csv`, whose five plans are genuinely imbalanced — a rare “Ultra” plan among four common ones:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import f1_score, classification_report, confusion_matrix
```

```
# plans.csv: which of five plans a customer upgraded to – a five-class target.
plans = pd.read_csv("plans.csv")
Xp = plans.drop(columns="plan")
yp = plans["plan"]
Xp_train, Xp_test, yp_train, yp_test = train_test_split(
    Xp, yp, test_size=0.2, random_state=0, stratify=yp)

softmax = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
ovr = make_pipeline(StandardScaler(),
                    OneVsRestClassifier(LogisticRegression(max_iter=1000)))
softmax.fit(Xp_train, yp_train)
ovr.fit(Xp_train, yp_train)

pred_softmax = softmax.predict(Xp_test)
pred_ovr = ovr.predict(Xp_test)
print("classes:", list(softmax.classes_))
print(f"softmax macro-F1: {f1_score(yp_test, pred_softmax, average='macro'):.2f}"
      f"    micro-F1: {f1_score(yp_test, pred_softmax, average='micro'):.2f}")
print(f"OvR macro-F1: {f1_score(yp_test, pred_ovr, average='macro'):.2f}"
      f"    micro-F1: {f1_score(yp_test, pred_ovr, average='micro'):.2f}")

# The K x K confusion matrix the per-class report is built from: rows are the
# actual plan, columns the predicted one, the diagonal correct.
cm = confusion_matrix(yp_test, pred_softmax, labels=softmax.classes_)
grid = pd.DataFrame(cm, index=softmax.classes_, columns=softmax.classes_)
grid.index.name = "actual\\pred"
print(grid)
print(classification_report(yp_test, pred_softmax, digits=2))
```

```
classes: ['Basic', 'Family', 'Premium', 'Standard', 'Ultra']
softmax macro-F1: 0.56 micro-F1: 0.73
OvR macro-F1: 0.53 micro-F1: 0.71
```

	Basic	Family	Premium	Standard	Ultra
actual\pred					
Basic	183	14	10	27	0
Family	28	61	6	26	0
Premium	15	6	69	13	0
Standard	30	14	10	268	0
Ultra	8	3	1	8	0
	precision		recall	f1-score	support
Basic	0.69		0.78	0.73	234
Family	0.62		0.50	0.56	121
Premium	0.72		0.67	0.69	103
Standard	0.78		0.83	0.81	322
Ultra	0.00		0.00	0.00	20
accuracy				0.73	800
macro avg	0.56		0.56	0.56	800
weighted avg	0.70		0.73	0.71	800

5.3. Evaluating multi-class models

The confusion matrix generalises to a $K \times K$ grid — the five-by-five table in the run above. Rows are the actual plan, columns the predicted one (the same axes as Figure 2), and the diagonal holds the correct predictions. Read down the **Ultra** row: its 20 customers are scattered across the four common plans' columns and its diagonal cell is 0; in fact the whole **Ultra** column is zero — the model never once predicts Ultra, so every real Ultra customer is misclassified. That empty diagonal cell is exactly why Ultra's per-class F1 is 0.00, and it is the fact the averages below turn on. Precision, recall and F1 are computed **per class** (treating that class as positive, the rest as negative) and then **averaged** — and how you average matters.

Definition 15. **Macro-averaging** computes the metric for each class and takes the plain mean, weighting every class equally. **Micro-averaging** pools all classes' TP, FP, FN into one calculation, weighting every **example** equally (so large classes dominate). **Weighted averaging** averages per-class metrics weighted by class size.

The run shows the divergence directly: micro-F1 (0.73) sits well above macro-F1 (0.56). Micro-averaging pools every example, so it is carried by the common plans the model serves well — **Standard** (F1 0.81) and **Basic** (F1 0.73), the two largest classes in the per-class report. Macro-averaging weights every class equally, so the rare **Ultra** plan — which the model never once predicts correctly (F1 0.00) — counts as much as any common plan and drags the average down. Micro flatters; macro exposes.

Pitfall. On imbalanced multi-class data the averaging choice changes the story. Micro-average is dominated by the big classes and can look excellent while a small but important class is handled terribly; macro-average exposes that by giving the rare class equal say. Choose the average to match what you care about — and always state which one you reported, because “F1 = 0.82” is ambiguous until you do.

Exercises

5.1. Train a multi-class classifier with one-vs-rest and with softmax on a genuinely multi-class target (three or more classes). Compare their predictions, build the $K \times K$ confusion matrix, and report macro- and micro-averaged F1, explaining which class drives the gap between the two averages and why.

Answers. (1) Macro- and micro-F1 agree when classes are balanced and diverge under imbalance — micro tracks the big classes, macro exposes a poorly served rare one; name the class whose weak per-class F1 opens the gap.

6. Feature scaling

We now turn from evaluation to the preprocessing judgment this course closes on, starting with scaling. Logistic regression sums $\beta_j x_j$, and the regularization penalty (next section) sums β_j^2 or $|\beta_j|$. Both are distorted when features live on wildly different numeric scales.

Example — when scale quietly skews a penalised model. Our four numeric features live on different scales: support tickets run 0–5, monthly price about 5–23, tenure 1–60 months, and watch hours 0–136 — the widest feature spans more than twenty times the narrowest. That gap alone makes the raw coefficients hard to read: a feature measured in large numbers needs only a small coefficient to exert its effect, while a small-range feature needs a large one, so coefficient size reflects **units** as much as importance. L2 regularization, which penalises $\sum \beta_j^2$, then takes those coefficients at face value — leaning hard on a small-range feature’s naturally large coefficient and barely touching a large-range feature’s naturally small one, constraining features by their units rather than their importance. Standardizing first puts every feature on one footing, so the penalty is fair.

Definition 16. Standardization rescales a feature to zero mean and unit variance:

$$x' = \frac{x - \mu}{\sigma},$$

using the feature’s training-set mean μ and standard deviation σ . Values become “number of standard deviations from the mean,” unbounded but centred.

Definition 17. Normalization (min–max scaling) rescales a feature to a fixed range, usually $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Values are bounded but sensitive to outliers, since a single extreme $x_{\max}$ compresses everyone else.

We can watch this on our own features — comparing the coefficients, the fit under a strong penalty, and the predictions with no penalty at all:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import roc_auc_score

Xr, Xr_test = X_train[numeric].to_numpy(), X_test[numeric].to_numpy()
scaler = StandardScaler().fit(Xr) # mean, std from the training set
only
Xs, Xs_test = scaler.transform(Xr), scaler.transform(Xr_test)

# (1) Coefficients. On raw features their sizes reflect units, not importance;
```

```

# after scaling each is an effect per one standard deviation, so comparable.
raw = LogisticRegression().fit(Xr, y_train)
scaled = LogisticRegression().fit(Xs, y_train)
print("raw coefficients:", raw.coef_[0].round(3))
print("scaled coefficients:", scaled.coef_[0].round(3))

# (2) Under a strong penalty the accident of units skews the fit: on raw features
# the L2 penalty constrains each feature by its numbers, not its importance.
raw_p = LogisticRegression(C=0.001, max_iter=5000).fit(Xr, y_train)
scaled_p = LogisticRegression(C=0.001, max_iter=5000).fit(Xs, y_train)
scale_auc_raw = roc_auc_score(y_test, raw_p.predict_proba(Xr_test)[: , 1])
scale_auc_scaled = roc_auc_score(y_test, scaled_p.predict_proba(Xs_test)[: , 1])
print(f"strong penalty AUC raw: {scale_auc_raw:.3f} scaled: {scale_auc_scaled:.3f}")

# (3) With no penalty at all, scaling is a pure reparametrization: same predictions.
opt = dict(C=np.inf, max_iter=100000, tol=1e-10)
raw_u = LogisticRegression(**opt).fit(Xr, y_train)
scaled_u = LogisticRegression(**opt).fit(Xs, y_train)
agree = np.mean(raw_u.predict(Xr_test) == scaled_u.predict(Xs_test))
print(f"un-regularized: raw and scaled agree on {agree:.0%} of test rows")

# (4) Normalization (min-max) is the other standard scaler: same scikit-learn API,
# but it maps each feature into [0, 1] rather than centring it. It, too, is a
# linear rescale, so the un-regularized model's predictions are unchanged.
mm = MinMaxScaler().fit(Xr)
Xm, Xm_test = mm.transform(Xr), mm.transform(Xr_test)
print("min-max ranges: min", Xm.min(0).round(2), " max", Xm.max(0).round(2))
minmax_u = LogisticRegression(**opt).fit(Xm, y_train)
agree_mm = np.mean(raw_u.predict(Xr_test) == minmax_u.predict(Xm_test))
print(f"un-regularized: raw and min-max agree on {agree_mm:.0%} of test rows")

```

```

raw coefficients: [-0.11 -0.048 0.059 1.827]
scaled coefficients: [-1.876 -1.005 0.261 1.608]
strong penalty AUC raw: 0.864 scaled: 0.901
un-regularized: raw and scaled agree on 100% of test rows
min-max ranges: min [0. 0. 0. 0.] max [1. 1. 1. 1.]
un-regularized: raw and min-max agree on 100% of test rows

```

Three things to read off the run. **The raw coefficients mislead:** support tickets' 1.83 towers over tenure's -0.11 only because tickets span a range of five and tenure a range of sixty; standardized, tenure (-1.88) is in fact the **largest** of the four effects, not the smallest. **The penalty is unfair on raw features:** under a strong L2 penalty the raw-feature model scores AUC 0.86 against the scaled model's 0.90, because the penalty falls on whichever feature carries big numbers rather than on whichever matters. **With no penalty the difference vanishes:** the un-regularized model predicts identically either way, because there scaling is a pure reparametrization. Scaling matters exactly when a penalty — or a distance, or gradient descent — makes the model care about the raw size of the coefficients.

The run's last two lines apply **normalization** to the same features for contrast: `MinMaxScaler` shares `StandardScaler`'s API but maps each feature into $[0, 1]$ instead of centring it — the four printed ranges all collapse to a clean 0-to-1. Being another linear rescale, it too leaves the un-regularized

model’s predictions identical; standardization and normalization differ in the numbers they hand a penalty, not in this invariance.

When does it matter? **Standardization** is the safe default for linear and logistic models, especially with regularization, and is more robust to outliers than min–max. **Normalization** suits features you want bounded (e.g. for certain neural nets or when a feature is already a bounded proportion). And crucially — **when does it not matter?**

Example — when scaling changes nothing — and when it still helps. A plain, **un-regularized** logistic regression is invariant to feature scaling **at its optimum**, as the run above shows: rescale a feature by a constant and the fitted coefficient simply rescales to compensate, leaving predictions unchanged. But reaching that optimum is another matter — the gradient descent of Section 2.3 crawls when features span wildly different magnitudes (the loss surface becomes a long, thin valley), so scaling still speeds and steadies the fit even with no penalty. That is why the un-regularized fit in the run sets a very tight `tol` — the solver’s **convergence tolerance**, the step size below which it stops — alongside a high `max_iter`: on badly-scaled features the solver would otherwise halt **short** of the optimum, and the scale-invariance holds only **at** the optimum, not before it. Decision trees and random forests are the genuinely scale-free case: they split one feature at a time on a threshold, and a monotone rescaling never changes the order. So scaling matters for **penalised** linear/logistic models, for **gradient-descent** fitting, and for **distance-based** methods; it is truly wasted effort only for trees. Knowing when **not** to scale is as much a part of the judgment as knowing when to.

Pitfall. Fit the scaler on the **training data only**, then apply its stored μ, σ (or $x_{\min}, x_{\max}$) to validation and test data. Computing the mean and standard deviation over the whole dataset leaks information from the held-out sets into training and inflates your scores — exactly the leak that scikit-learn pipelines, from *Intro to Machine Learning*, exist to prevent. Put the scaler **inside** the pipeline.

The pipeline is what makes “fit on training only” automatic: `StandardScaler` learns its μ, σ on each fold’s training part and re-applies them, no leak possible.

```
from sklearn.preprocessing import StandardScaler # or MinMaxScaler for [0, 1]
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train[numeric], y_train) # scaler fitted on X_train only, inside the
pipeline
```

Exercises

6.1. Apply standardization and min–max normalization to features on different scales. Show that an L2-regularized logistic regression’s coefficients change with scaling, and that an un-regularized one’s ($C=\text{np.inf}$) predictions do not. To see the invariance you must let the solver actually reach the optimum: on badly-scaled features it stops early, so raise `max_iter` and tighten `tol` until the scaled and unscaled predictions agree — and note that needing to do so **is** the scaling lesson. State when scaling is necessary and when it is irrelevant.

Answers. **(1)** The L2-regularized coefficients change under scaling; the un-regularized model's predictions do not — **provided the solver converges**, so tighten `tol`/raise `max_iter` if the scaled and unscaled fits disagree, since badly-scaled features stop the optimiser short. Scaling is necessary for penalised or distance-based models and for fast, reliable convergence; it is irrelevant to the predictions of a **converged** un-penalised linear/logistic model and to trees.

7. Feature engineering for linear models

A linear model can only draw straight boundaries in the features you give it. Its power therefore lives largely in **which features you build**. Three constructions do most of the work.

7.1. Polynomial features

Definition 18. **Polynomial features** add powers of existing features ($x, x^2, x^3, \dots$) so a linear model can fit curved relationships. The model stays linear **in its coefficients** — it is still $\beta_0 + \beta_1 x + \beta_2 x^2$ — but now bends as a function of x .

This is exactly the polynomial regression of *Intro to Machine Learning*, reframed as a **feature construction**: you are not changing the model, you are enriching its inputs. The same overfitting risk applies — high-degree features fit noise — so they pair naturally with regularization and cross-validated degree selection.

`PolynomialFeatures` builds them for you — and, usefully, `degree=2` already emits the interaction terms of the next section (the cross-products) alongside the squares:

```
from sklearn.preprocessing import PolynomialFeatures

# [tenure, watch_hours] -> squares AND the cross-term tenure*watch_hours
poly = PolynomialFeatures(degree=2, include_bias=False)
poly.fit(X_train[["tenure", "watch_hours"]])
print(list(poly.get_feature_names_out()))
```

```
['tenure', 'watch_hours', 'tenure^2', 'tenure watch_hours', 'watch_hours^2']
```

7.2. Interaction terms

Example — when two features only matter together. Among your subscribers, a high price alone is tolerable and low watch hours alone is tolerable, but a customer paying a **high price** who **barely watches** is a churn risk far beyond what either feature predicts on its own. A plain linear model, adding the two effects separately, cannot express “high price **and** low usage.” The **product** of the two features can.

Definition 19. An **interaction term** is a new feature formed by multiplying two features, $x_i \times x_j$. It lets a linear model capture effects that depend on **combinations** of features — situations where the impact of one feature changes with the value of another.

The `PolynomialFeatures` of the previous section would manufacture **every** cross-product at once; more often you want one **specific**, domain-chosen interaction. That is simply one column times another — here the price-times-usage feature the example calls for:

```
# One deliberately chosen interaction, motivated by the domain: does a high
# price bite harder when a subscriber barely watches? Build that single feature
# by multiplying the two raw columns, and add it alongside the originals.
X_ix = X_train[numeric].copy()
X_ix["price_x_watch"] = X_ix["monthly_price"] * X_ix["watch_hours"]
print(X_ix[["monthly_price", "watch_hours", "price_x_watch"]].head(4).round(1))
```

	monthly_price	watch_hours	price_x_watch
2078	8.7	11.8	102.4
4844	8.8	135.6	1194.6
5163	9.2	56.7	521.1
5315	9.4	15.3	143.8

The new `price_x_watch` column now sits beside the originals as an ordinary feature, and the model gives it its own coefficient — letting the effect of price on churn depend on how much the subscriber watches, which separate price and watch-hours terms cannot express.

7.3. Encoding categorical variables

Linear models consume numbers, but features like **subscription plan** (Basic / Standard / Premium) or **country** are categories. They must be encoded — and the naive encoding is a trap.

Definition 20. One-hot encoding turns a categorical feature with K levels into K binary (0/1) indicator features, exactly one of which is 1 per row. **Label (ordinal) encoding** maps the levels to integers 0, 1, 2,

Pitfall. Do not label-encode an **unordered** category. Mapping Basic→0, Standard→1, Premium→2 tells the model Premium is “three Basics” and that Standard sits exactly between — a false ordering it will dutifully exploit, fitting nonsense. Use one-hot encoding for unordered categories; reserve integer/ordinal encoding for genuinely ordered ones (e.g. small < medium < large). With many levels, one-hot can explode the feature count, which is itself a reason to lean on L1 regularization next.

`OneHotEncoder` does the safe thing; a `ColumnTransformer` lets you one-hot the categorical columns while leaving the numeric ones untouched, all in one object. On our table it turns two categorical columns into their indicator columns and passes the numeric ones through:

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

# handle_unknown="ignore": an unseen category becomes all-zeros (no crash)
pre = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["plan", "country"]),
    ("num", "passthrough", ["tenure", "watch_hours"]), # numeric columns untouched
])
encoded = pre.fit_transform(X_train)
```

```
print("columns after encoding:", encoded.shape[1])
print(list(pre.get_feature_names_out()))
```

```
columns after encoding: 9
['cat__plan_Basic', 'cat__plan_Premium', 'cat__plan_Standard', 'cat__country_A',
'cat__country_B', 'cat__country_C', 'cat__country_D', 'num__tenure',
'num__watch_hours']
```

Remark. Full one-hot encoding gives a feature K indicator columns that always sum to 1, so together with the model's intercept they are exactly collinear — the **dummy-variable trap**, an instance of the multicollinearity you learned to diagnose in *Intro to Machine Learning*. It is harmless under the regularized LogisticRegression used throughout this book, which is why we keep all K columns; but if you ever fit an unregularized model or read the raw coefficients, drop one level per feature (`OneHotEncoder(drop="first")`) so the remaining columns are no longer redundant.

Exercises

7.1. Engineer features for a linear model: add polynomial terms, an interaction term motivated by the domain, and one-hot encode a categorical variable. Show a case where label-encoding an unordered category produces a worse model, and explain why.

Answers. (1) Label-encoding an unordered category injects a false order and degrades the fit versus one-hot; a well-motivated interaction term should improve it.

8. Regularization for logistic regression

Regularization carries over from *Intro to Machine Learning* essentially unchanged — the same L1 and L2 penalties, added now to the log loss instead of the RSS — and the same intuition for which to prefer. It matters more here because the feature engineering above can multiply your feature count (polynomials, interactions, one-hot levels) and invite overfitting.

Definition 21. Regularized logistic regression adds a coefficient penalty to the log loss:

$$L_{L2} = \log \text{ loss} + \lambda \sum_{j=1}^p \beta_j^2, \quad L_{L1} = \log \text{ loss} + \lambda \sum_{j=1}^p |\beta_j|.$$

As in linear models, **L1 (lasso)** drives some coefficients exactly to zero — **automatic feature selection** — while **L2 (ridge)** shrinks coefficients smoothly toward zero for **stability**, never quite eliminating them.

The choice mirrors the one you already know. Prefer **L1** when you suspect many of your engineered features are useless and you want a sparse, interpretable model that names the few that matter — invaluable after a one-hot or polynomial expansion has produced hundreds of candidate features. Prefer **L2** when you believe most features contribute a little and you mainly want to keep coefficients stable, especially when features are correlated (as interaction and polynomial terms inevitably are with their parents).

Pitfall. scikit-learn's `LogisticRegression` is regularized **by default** (L2, strength set by `C`, where $C = \frac{1}{\lambda}$ — so **smaller** `C` means **stronger** regularization). This is easy to miss: the model you get “out of the box” is already penalised. Set `C` deliberately and tune it by cross-validation, as you tuned λ for ridge. And — as everywhere — scale features before penalising them, or the penalty falls unevenly.

You pick the penalty with `penalty=` (L1 needs the `liblinear` or `saga` solver) and let `LogisticRegressionCV` search `C` for you, scoring each candidate by cross-validation on the metric you chose. Here we hand it a deliberately wide, partly-redundant feature set (degree-2 polynomials of the numeric columns plus the one-hot categories) and let L1 prune it:

```
from sklearn.linear_model import LogisticRegressionCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.pipeline import make_pipeline

# A deliberately wide feature set: degree-2 polynomials of the numeric columns
# (squares + interactions) plus one-hot categories — many of them redundant.
engineer = ColumnTransformer([
    ("num", make_pipeline(StandardScaler(),
                        PolynomialFeatures(degree=2, include_bias=False)), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
Z_train = engineer.fit_transform(X_train)
```

```
model = LogisticRegressionCV(  
    Cs=10, cv=5, penalty="l1", solver="liblinear", scoring="f1",  
).fit(Z_train, y_train)  
  
print(f"features: {Z_train.shape[1]}")  
print(f"C picked by CV: {model.C_[0]:.4f}")  
print(f"coefficients L1 drove to zero: {(model.coef_[0] == 0).sum()}")
```

```
features: 21  
C picked by CV: 2.7826  
coefficients L1 drove to zero: 2
```

L1 drove two of the engineered coefficients to exactly zero — a small selection here, but on a wider expansion (hundreds of polynomial and one-hot columns) this is how a model names the few features that carry the signal.

Exercises

8.1. Apply L1 and L2 regularization to a logistic regression with many engineered features. Report which coefficients L1 drives to zero, tune C by cross-validation, and give a situation where each penalty is the better choice.

Answers. (1) L1 zeros out coefficients (report the count); L2 keeps them small but non-zero; tune C by cross-validation. Prefer L1 to select from many possibly useless features, L2 for stability among correlated ones.

9. When simple models win

You could reach for a gradient-boosted forest or a neural network for churn, and sometimes you should. But this course argues, deliberately, that logistic regression is often the **right** model — not merely the easy one — and naming why is part of the judgment this course asks of you.

Interpretability. A logistic regression hands you a coefficient per feature: “each support ticket raises the odds of churn several-fold.” When the retention team, a regulator, or a sceptical executive asks **why** a customer was flagged, you can answer in a sentence. A deep model’s answer is “the weights say so.” In regulated domains (credit, insurance, hiring) this is not a nicety — an unexplainable decision can be illegal.

Data efficiency. With a few thousand rows, a simple model with few parameters generalises better than a flexible one starved of data. Complex models need large data to earn their flexibility; below that, they overfit and a logistic regression wins outright.

Inference speed and operational cost. A logistic regression scores a customer with one dot product and one sigmoid — microseconds, trivial to deploy, easy to monitor, cheap to retrain. A large model can cost orders of magnitude more to serve at scale and is far harder to debug when it drifts.

Example — when the simpler model is the better choice. A boosted forest scores AUC 0.93 against logistic regression’s 0.91. The forest has the higher score. But the logistic model can be explained to the compliance team, retrained nightly on a laptop, and its flagged reasons given to the retention team as concrete talking points. On a two-point AUC gap, the simpler model that the whole organisation can understand, audit and operate is frequently the better **business** decision. “Best model” is not “highest score” — it is best given accuracy, interpretability, data and cost together.

Pitfall. The converse error is real too: never reach for complexity **by reflex**. The right order is to make the simple model work, evaluate it honestly with the toolkit above, and adopt a complex model only when the measured gain justifies the cost in interpretability and operations. Complexity is a cost you pay, not a virtue you earn.

Exercises

9.1. You must choose between a logistic regression (AUC 0.86, one coefficient per feature) and a gradient-boosted forest (AUC 0.90, opaque) for a churn model that must be explained to a regulator, retrained nightly, and whose flagged reasons are given to the retention team. Argue which model you would deploy, weighing the AUC gap against interpretability, data efficiency and operational cost — and state what would change your choice.

Answers. (1) Deploy the logistic regression: a 4-point AUC gap rarely outweighs a model the regulator can be shown, the team can retrain cheaply, and whose per-feature reasons become talking points; what would flip the choice is a materially larger, well-validated performance gap, or a setting where interpretability and operational cost do not bind. “Best model” weighs accuracy, interpretability, data and cost together.

10. The end-to-end classification pipeline

Everything in this book assembles into one reproducible workflow — the scikit-learn pipeline of *Intro to Machine Learning*, now carrying a classifier and the classification evaluation toolkit. The shape of a complete classification project:

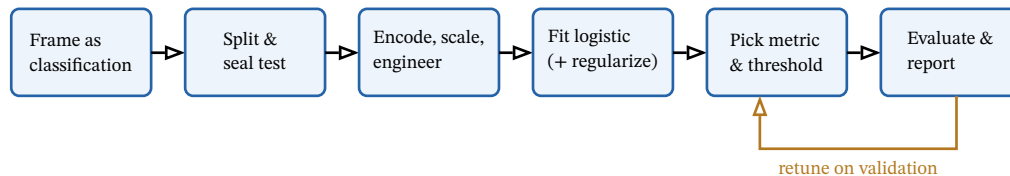


Figure 4: The end-to-end classification pipeline. Preprocessing (encode, scale, engineer) and the model live inside one scikit-learn pipeline so every step is fitted on training data only and re-applied identically under cross-validation and at test time. The metric and threshold are chosen on validation from the business costs; the test set is opened once, at the end, for the report.

In code, the whole preprocessing-plus-model stack is one Pipeline, so cross-validation re-fits every step — encoder, scaler, classifier — on each fold’s training part alone. This is the capstone template the final exercise asks you to build:

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pre = ColumnTransformer([
    ("num", StandardScaler(), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
])
pipe = Pipeline([
    ("pre", pre),
    ("clf", LogisticRegression(penalty="l2", C=1.0, max_iter=1000)),
])

# honest CV estimate: each fold re-fits every step on its own training part
scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring="f1")
print(f"cross-validated F1: {scores.mean():.3f} +/- {scores.std():.3f}")

pipe.fit(X_train, y_train)  # then fit on all the training data
p_hat = pipe.predict_proba(X_test)[: , 1]  # open the test set once, at the end
  
```

```
cross-validated F1: 0.561 +/- 0.039
```

The deliverable, though, is not that number but a **business-oriented evaluation report**: which metric you chose and why (in terms of the cost of a false positive versus a false negative), the confusion matrix translated into plain language, the operating threshold and what it buys, and an honest statement of what the model gets wrong. Here is a short specimen, written for our churn model from the numbers in this book — the genre the final exercise asks you to produce.

Example — a specimen evaluation report. Model. Regularized logistic regression on subscriber features, chosen for its interpretability (each feature carries a readable odds ratio) and low operating cost; cross-validated F1 on the training data was 0.56.

Metric choice. We report on **recall** first. A missed churner costs the customer’s whole lifetime value, while a false alarm costs one retention call — perhaps a hundred times less — so we care most about how many real leavers we catch, and read precision alongside it to keep wasted calls in view.

Results in plain language. On a held-out sample of 1200 subscribers, of every 100 customers who really left, the model caught 48 and of every 100 customers it flagged for a call, 69 had genuinely intended to leave. Put any real leaver next to a loyal customer and the model gives the leaver the higher risk score 91 times out of 100, so its **ranking** is strong even where its yes/no calls at the default cut are cautious.

Operating point. We run at a threshold of 0.5 for now. Lowering it to 0.3 would raise recall from 0.48 to 0.62 — catching more leavers — at the cost of dropping precision to 0.56 and roughly half again as many calls. We recommend that trade once the retention team confirms the call budget.

What it gets wrong. At the current cut the model still misses just over half of all churners, and one call in three lands on a customer who was not leaving. It should be retrained monthly and never used to **deny** anyone service — only to prioritise who to contact.

That report — model plus justified evaluation plus clear communication — is the whole course in one artefact.

Exercises

10.1. Build a complete scikit-learn pipeline (encode + scale + engineer + regularized logistic regression), evaluate it under cross-validation with a justified metric and threshold, and produce a one-page business-oriented evaluation report covering the confusion matrix in plain language, the ROC/AUC as a ranking statement, the operating threshold and its trade-off, and what the model gets wrong.

Answers. (1) Your report should name the metric and justify it by the cost of a false positive versus a false negative, give the confusion matrix in plain language, state the AUC as a ranking chance and the ROC as a menu of operating points, give the operating threshold and its trade-off, and admit what the model gets wrong; `cross_val_score` gives the honest performance estimate.

One idea has run under everything here. Every tool in this book followed from a single change of question — from regression’s “how much?” to “which?” and “yes or no?”. Make the target a category and three things follow that do not follow in regression: the model must emit a **probability** rather than a number, the two ways of being wrong stop being interchangeable, and the **threshold** that turns the probability into a verdict becomes a business decision rather than a default. That is why the model itself was the easy part — logistic regression is linear regression’s twin — and why the weight of the course fell on **evaluation**: choosing, and defending to the people who act on it, how you measure a classifier against the asymmetric costs of the decision it serves. Carry that instinct — measure it against the decision — into every model you meet after this one.

Sources and further reading

- Cox, D. R. (1958), “The Regression Analysis of Binary Sequences” — the paper that introduced logistic regression.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009), **The Elements of Statistical Learning**, 2nd ed. — Chapter 4 (logistic regression) and Chapter 7 (evaluation).
- Fawcett, T. (2006), “An Introduction to ROC Analysis,” **Pattern Recognition Letters** — the standard reference on ROC curves and AUC.
- The scikit-learn user guide, sections **Logistic regression**, **Preprocessing data**, and **Metrics and scoring** — the documentation for every tool used in this book.